

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



BÁO CÁO TIỂU LUẬN
ĐỀ TÀI: TIỂU LUẬN FRACTAL

GVHD: TS. Cáp Phạm Đình Thăng

Lớp: Đồ họa máy tính – CS105.Q22

- | | |
|-----------------------|----------------|
| 1. Lê Nguyễn Quốc Bảo | MSSV: 23520108 |
| 2. Phạm Khương Duy | MSSV: 23520383 |
| 3. Mai Xuân Tuấn | MSSV: 23521714 |

Thành phố Hồ Chí Minh – Thứ 4 ngày 8 tháng 4, 2026

1. Yêu cầu Tiểu luận:

- Tìm hiểu về cách vẽ bông tuyết Vankoch (Koch Snowflake).
- Tìm hiểu về cách vẽ đảo Minkowski.
- Tìm hiểu về cách vẽ tam giác Sierpinski (Triangles) và Hình vuông Sierpinski (Carpet).
- Tìm hiểu và trình bày tập Mandelbrot và Julia Set.

2. Nội dung Trình bày:

2.1. Bông tuyết Koch (Koch Snowflake):

2.1.1. Tổng quan về Bông tuyết Koch:

Bông tuyết Koch (Koch Snowflake) là một trong những fractal cổ điển nhất, được nhà toán học người Thụy Điển Helge von Koch giới thiệu năm 1904. Đây là một đường cong fractal nổi tiếng với đặc điểm:

- Chu vi vô hạn nhưng diện tích hữu hạn.
- Tính tự đồng dạng (self-similarity) hoàn hảo ở mọi cấp độ phóng đại.
- Là một đường cong liên tục nhưng không khả vi tại mọi điểm.

Cấu trúc này thể hiện rõ nét bản chất nghịch lý của toán học fractal: một hình có chu vi tiến tới vô cùng nhưng lại nằm gọn trong một diện tích giới hạn.

2.1.2. Quy trình xây dựng:

Bông tuyết Koch được tạo ra bằng cách bắt đầu từ một **tam giác đều**, sau đó thay đổi đệ quy từng đoạn thẳng theo các bước:

Bước 1: Chia đoạn thẳng thành ba đoạn bằng nhau.

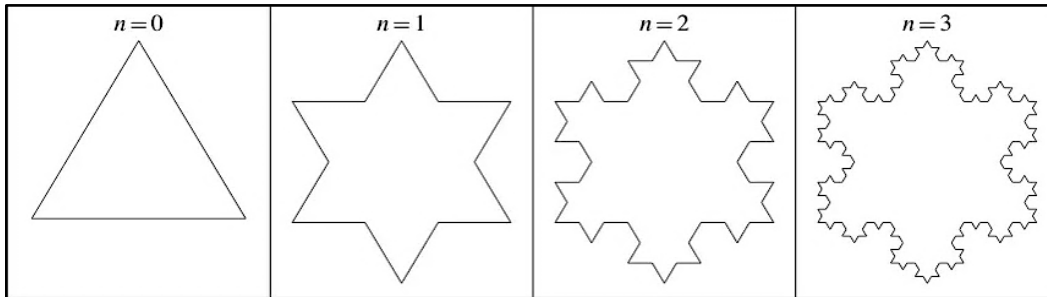
Bước 2: Vẽ một tam giác đều có cạnh là đoạn thẳng ở giữa (bước 1), hướng ra phía ngoài.

Bước 3: Loại bỏ đoạn thẳng là cạnh đáy của tam giác vừa vẽ ở bước 2.

* Lưu ý kỹ thuật:

- Lần lặp đầu tiên (Iteration 1) sẽ tạo ra hình dáng của một **ngôi sao 6 cánh (Hexagram)**.

- Băng tuyết Koch hoàn chỉnh là giới hạn của quy trình này khi số lần lặp tiến đến vô hạn.
- Về mặt cấu tạo, một băng tuyết Koch được ghép lại từ 3 **đường cong Koch**.



2.1.3. Các thuộc tính toán học:

- **Chu vi (Perimeter):** Sau mỗi lần lặp, chu vi tăng theo hệ số $4/3$. Gọi s là độ dài cạnh tam giác ban đầu, chu vi ở bước n là:

$$P_n = 3s \cdot \left(\frac{4}{3}\right)^n$$

Khi n tiến đến vô cùng, chu vi tiến đến **vô hạn**.

- **Diện tích (Area):** Mặc dù chu vi vô hạn, diện tích băng tuyết lại hữu hạn. Gọi A là diện tích tam giác đều ban đầu, diện tích sau n lần lặp là:

$$A_n = A \cdot \left(1 + \frac{3}{4} \sum_{k=1}^n \left(\frac{4}{9}\right)^k\right)$$

Khi tiến đến vô hạn, diện tích băng tuyết bằng **$8/5$** diện tích tam giác ban đầu:

$$A_{snowflake} = \frac{8}{5}A = \frac{2s^2\sqrt{3}}{5}$$

- **Chiều Hausdorff (Fractal Dimension):** Đường cong Koch có chiều fractal là:

$$D = \frac{\ln 4}{\ln 3} \approx 1.26186$$

Số chiều này lớn hơn đường thẳng (1D) nhưng nhỏ hơn đường cong lấp đầy không gian của Peano (2D).

- **Tính chất đặc biệt:** Tại bất kỳ điểm nào trên đường cong, ta **không thể vẽ được tiếp tuyến**. Điều này có nghĩa là hàm số tạo nên đường cong này liên tục nhưng không có đạo hàm tại bất kỳ đâu.

2.1.4. Xây dựng thuật toán

Thuật toán Koch được chia làm hai hàm chính: hàm khởi tạo hình dáng ban đầu và hàm đệ quy chia nhỏ các cạnh.

a. Khởi tạo hình đa giác gốc (getKochCoords)

- **Mục đích:** Tạo ra một tam giác đều làm hình gốc (level 0) cho bông tuyết.
- **Logic:** Sử dụng vòng lặp for lặp 3 lần để tính toán 3 đỉnh của một tam giác đều dựa trên lượng giác (Math.cos , Math.sin) với bán kính $r = 1.0$.
 - Sau đó, một vòng lặp for khác sẽ duyệt qua từng cạnh của tam giác (nối từ đỉnh i đến đỉnh $(i + 1) \% 3$).
 - Gọi hàm đệ quy kochPoints cho từng cạnh để sinh ra các điểm chi tiết của Fractal.
 - **Lưu ý kỹ thuật:** Mảng kết quả cuối cùng ghép thêm điểm đầu tiên (pts.push(pts[0])) để đảm bảo đường vẽ khép kín.

```
function getKochCoords(level) {
  const r = 1.0, angleOffset = Math.PI / 2;
  const vertices = [];
  for (let i = 0; i < 3; i++) {
    vertices.push({ x: r * Math.cos(angleOffset + i * 2 * Math.PI / 3), y: r * Math.sin(angleOffset + i * 2 * Math.PI / 3) });
  }
  let pts = [];
  for (let i = 0; i < 3; i++) {
    pts.push(...kochPoints(vertices[i], vertices[(i + 1) % 3], level).slice(0, -1));
  }
  pts.push(pts[0]);
  return pts;
}
```

b. Đệ quy sinh điểm chi tiết (kochPoints)

- **Điều kiện dừng:** Nếu $\text{level} == 0$, hàm trả về mảng chứa đúng 2 điểm đầu và cuối của đoạn thẳng $[p1, p2]$.
- **Logic cắt đoạn:** Nếu $\text{level} > 0$, đoạn thẳng nối từ $p1$ đến $p2$ sẽ được chia làm 3 phần bằng nhau.

- Điểm a: Nằm ở vị trí 1/3 đoạn thẳng.
- Điểm b: Nằm ở vị trí 2/3 đoạn thẳng.
- **Tìm đỉnh chóp:** Thuật toán sử dụng phép xoay vector để tìm điểm peak tạo thành một tam giác đều nhô ra ngoài giữa đoạn a và b. Góc xoay được sử dụng là $-\text{Math.PI} / 3$ (tức -60 độ).
- **Gọi đệ quy và Gộp mảng:** Hàm tiếp tục gọi chính nó cho 4 đoạn thẳng mới được tạo ra: $(p1, a)$, (a, peak) , (peak, b) , và $(b, p2)$ với $\text{level} - 1$.
- **Xử lý trùng lặp:** Ở mỗi lần gộp mảng, thuật toán sử dụng `.slice(0, -1)` để cắt bỏ điểm cuối của các đoạn trước. Điều này rất quan trọng để tránh việc lưu trữ trùng lặp các đỉnh đứng liền kề nhau.

```
function kochPoints(p1, p2, level) {
  if (level === 0) return [p1, p2];
  const dx = p2.x - p1.x, dy = p2.y - p1.y;
  const a = { x: p1.x + dx / 3, y: p1.y + dy / 3 };
  const b = { x: p1.x + 2 * dx / 3, y: p1.y + 2 * dy / 3 };
  const angle = -Math.PI / 3;
  const peak = {
    x: a.x + (b.x - a.x) * Math.cos(angle) - (b.y - a.y) * Math.sin(angle),
    y: a.y + (b.x - a.x) * Math.sin(angle) + (b.y - a.y) * Math.cos(angle)
  };
  return [...kochPoints(p1, a, level - 1).slice(0, -1),
    ...kochPoints(a, peak, level - 1).slice(0, -1),
    ...kochPoints(peak, b, level - 1).slice(0, -1),
    ...kochPoints(b, p2, level - 1)];
}
```

2.2. Đảo Minkowski (Minkowski Island):

2.2.1. Tổng quan về đảo Minkowski:

Đảo Minkowski (Minkowski Island), hay đôi khi được gọi là Xúc xích Minkowski (Minkowski Sausage) ở dạng đường cong hở, là một cấu trúc fractal hình học độc đáo được đặt theo tên của nhà toán học Hermann Minkowski. Khác với Koch bắt đầu từ tam giác, Đảo Minkowski thường được khởi tạo từ một hình vuông cơ sở và mang các đặc điểm nổi bật sau:

- **Chu vi vô hạn trong một diện tích hữu hạn:** Quá trình đệ quy tạo ra những đường gấp khúc lồi lõm liên tục, làm cho chiều dài đường viền tiến tới vô cực nhưng toàn bộ hình khối vẫn bị giới hạn trong không gian ban đầu.

- **Tính tự đồng dạng (self-similarity):** Mỗi phân đoạn nhỏ trên đường viền đều chứa cấu trúc ziczac lặp lại giống hệt với hình dáng của toàn bộ fractal ở các cấp độ lớn hơn.
- **Tốc độ gia tăng phức tạp cực cao:** Tại mỗi bước lặp, một đoạn thẳng bị chia cắt và thay thế bằng 8 đoạn thẳng nhỏ hơn (hệ số nhân 8), khiến số lượng đỉnh tăng vọt nhanh chóng hơn nhiều so với Băng tuyết Koch.

Cấu trúc này minh họa xuất sắc cho việc áp dụng một quy tắc rẽ nhánh lùi - lõm đơn giản để sinh ra một đường viền có độ phức tạp hình học khổng lồ. Trong đồ họa máy tính, Đảo Minkowski thường được sử dụng để đánh giá hiệu suất xử lý mảng đỉnh (vertex arrays) của thuật toán đệ quy.

2.2.2. Quy trình xây dựng:

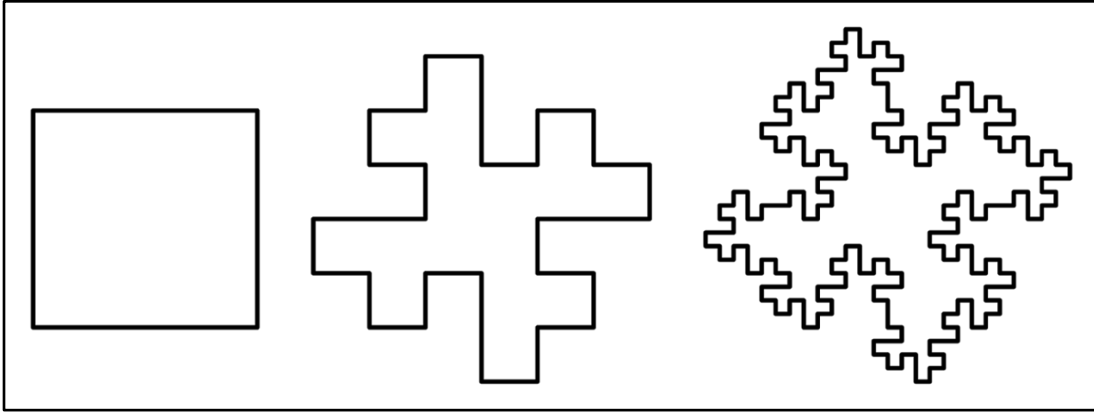
Đảo Minkowski là một hệ động lực fractal dựa trên cấu trúc hình vuông, được xây dựng bằng cách thay thế đệ quy các cạnh của một hình vuông ban đầu theo một quy tắc thay thế (generator) cụ thể:

Bước 0: Bắt đầu với một **hình vuông** đơn vị (còn gọi là vật khởi đầu - initiator).

Bước 1 (Quy tắc thay thế): Chia mỗi cạnh của hình vuông thành **4 đoạn thẳng** bằng nhau. Thay thế mỗi cạnh bằng một đường gấp khúc gồm **8 đoạn thẳng** nhỏ (mỗi đoạn có chiều dài bằng $1/4$ cạnh ban đầu).

Cấu trúc đường gấp khúc: Quy tắc này tạo ra các góc vuông (90°) hướng vào trong và ra ngoài xen kẽ, tạo nên hình dáng giống như một sơ đồ răng cưa hoặc các "vịnh" và "bán đảo" vuông vức.

Bước n: Lặp lại quy trình trên vô hạn lần cho mọi phân đoạn mới phát sinh.



2.2.3. Các thuộc tính toán học:

Giống như Băng tuyết Koch, đảo Minkowski sở hữu những đặc tính kỳ lạ của hình học fractal:

- **Chu vi (Perimeter):** Ở mỗi bước lặp, một đoạn thẳng độ dài L được thay thế bởi 8 đoạn thẳng độ dài $L/4$. Do đó, tổng chiều dài tăng gấp đôi ($8 \times 1/4 = 2$) sau mỗi lần lặp.

$$P_n = P_0 \cdot 2^n$$

Khi n tiến đến vô cùng, chu vi của đảo Minkowski tiến đến **vô hạn**.

- **Diện tích (Area):** Mặc dù chu vi tiến đến vô hạn, diện tích của đảo Minkowski vẫn **hữu hạn** và bị giới hạn bên trong một vùng không gian nhất định trên mặt phẳng. Điều này minh chứng cho nghịch lý fractal: một đường biên dài vô tận bao quanh một diện tích hữu hạn.

- **Chiều Hausdorff (Fractal Dimension):** Đây là thông số quan trọng nhất để phân biệt với Koch. Với $N = 8$ (số đoạn mới) và $1/r = 4$ (tỷ lệ chia đoạn), chiều fractal được tính bằng:

$$D = \frac{\log(8)}{\log(4)} = 1.5$$

Ý nghĩa: Với $D = 1.5$, đảo Minkowski có độ phức tạp và khả năng lấp đầy mặt phẳng cao hơn đáng kể so với đường cong Koch ($D \approx 1.26$). Nó nằm chính giữa ranh giới của một đường thẳng (1D) và một mặt phẳng (2D).

2.2.4. Xây dựng thuật toán:

Tương tự như Koch, thuật toán Minkowski cũng dựa trên nguyên lý khởi tạo hình gốc và đệ quy thay thế các đoạn thẳng.

a. Khởi tạo hình đa giác gốc (getMinkowskiCoords)

- **Mục đích:** Tạo ra một hình vuông gốc (thay vì tam giác như Koch).
- **Logic:** Khởi tạo thủ công 4 đỉnh của một hình vuông có độ dài cạnh được chuẩn hóa dựa trên $h = 1.0 / \text{Math.sqrt}(2)$.
- Vòng lặp for duyệt qua 4 cạnh của hình vuông và gọi đệ quy `minkowskiPoints` để lấy tọa độ các điểm. Cuối cùng, điểm đầu tiên được nối lại vào cuối mảng để khép kín hình.

```
function getMinkowskiCoords(level) {  
  const h = 1.0 / Math.sqrt(2);  
  const vertices = [{ x: h, y: -h }, { x: h, y: h }, { x: -h, y: h }, { x: -h, y: -h }];  
  let pts = [];  
  for (let i = 0; i < 4; i++) pts.push(...minkowskiPoints(vertices[i], vertices[(i + 1) % 4], level).slice(0, -1));  
  pts.push(pts[0]);  
  return pts;  
}
```

b. Đệ quy sinh điểm chi tiết (minkowskiPoints)

- **Điều kiện dừng:** Nếu `level == 0`, trả về 2 điểm đầu cuối `[p1, p2]`.
- **Logic cắt đoạn:** Đoạn thẳng `[p1, p2]` được chia theo các tỷ lệ 1/4.
 - `fx, fy`: Khoảng cách dịch chuyển tịnh tiến 1/4 độ dài đoạn thẳng.
 - `ox, oy`: Vector pháp tuyến (vuông góc) dùng để tạo độ "gấp khúc" (bằng cách đảo chiều và đổi dấu `dx, dy`).
- **Kịch bản thay thế:** Thuật toán Minkowski thay thế 1 đoạn thẳng bằng 8 đoạn nhỏ liên tiếp nhau. Mã nguồn định nghĩa một mảng chứa 8 bước dịch chuyển tương đối:
 - Tiến lên (`[fx, fy]`)
 - Rẽ trái vuông góc (`[ox, oy]`)
 - Tiến lên (`[fx, fy]`)
 - Rẽ phải vuông góc đi xuống (`[-ox, -oy]`) 2 lần để đi xuyên qua trục chính.
 - Tiến lên (`[fx, fy]`)
 - Rẽ trái vuông góc (`[ox, oy]`) để về lại trục chính.

- Tiến lên ($[fx, fy]$) đến điểm đích.
- **Xử lý vòng lặp:** Hàm dùng `forEach` duyệt qua 8 bước dịch chuyển này, cộng dồn tọa độ để sinh ra danh sách các điểm mới. Sau đó, dùng vòng lặp `for` để tiếp tục gọi đệ quy `minkowskiPoints` cho từng đoạn nối giữa các điểm mới này với `level - 1`. Cuối cùng, gộp kết quả và lược bỏ các đỉnh trùng lặp bằng `.slice(0, -1)`.

```
function minkowskiPoints(p1, p2, level) {
  if (level === 0) return [p1, p2];
  const dx = p2.x - p1.x, dy = p2.y - p1.y;
  const fx = dx / 4, fy = dy / 4, ox = dy / 4, oy = -dx / 4;
  let cur = { x: p1.x, y: p1.y };
  const pts = [cur];
  [[fx, fy], [ox, oy], [fx, fy], [-ox, -oy], [-ox, -oy], [fx, fy], [ox, oy], [fx, fy]].forEach(m => {
    cur = { x: cur.x + m[0], y: cur.y + m[1] };
    pts.push(cur);
  });
  const result = [];
  for (let i = 0; i < pts.length - 1; i++) result.push(...minkowskiPoints(pts[i], pts[i + 1], level - 1).slice(0, -1));
  result.push(pts[pts.length - 1]);
  return result;
}
```

2.3. Tam giác Sierpinski (Triangles) và Hình vuông Sierpinski (Carpet):

Trong phần này, nhóm trình bày hai cấu trúc fractal tiêu biểu:

- **Tam giác Sierpinski (Sierpinski Triangle):** được xây dựng từ một tam giác đều bằng cách loại bỏ tam giác ở trung tâm.
- **Thảm Sierpinski (Sierpinski Carpet):** mở rộng ý tưởng trên sang hình vuông, sử dụng phép chia lưới 3×3 và loại bỏ ô trung tâm.

Hai cấu trúc này đều thể hiện rõ tính **tự đồng dạng** – đặc trưng cốt lõi của fractal.

2.3.1. Tam giác Sierpinski (Sierpinski Triangle):

2.3.1.1. Định nghĩa và phương pháp xây dựng:

Tam giác Sierpinski được xây dựng theo quy trình đệ quy:

1. **Bước 0:** Khởi tạo một tam giác đều ban đầu.
2. **Bước 1:** Xác định trung điểm của ba cạnh, nối các điểm này để tạo thành một tam giác nhỏ ở giữa, sau đó loại bỏ tam giác này. Kết quả thu được ba tam giác con tại ba góc.

3. **Bước n:** Lặp lại quy trình trên cho từng tam giác con còn lại.

Khi số bước tiến tới vô hạn, ta thu được một cấu trúc fractal hoàn chỉnh.

2.3.1.2. Số lượng tam giác và số đỉnh:

Số lượng tam giác tăng theo cấp số nhân với cơ số 3:

Bước (n):	Số tam giác:	Số đỉnh (WebGL):
0	1	3
1	3	9
2	9	27
n	3^n	3^{n+1}

2.3.1.3. Thuật toán đệ quy:

Trong quá trình đệ quy, tam giác ở trung tâm (tam giác được tạo từ các trung điểm) không được tiếp tục xử lý, tức là không được vẽ. Điều này tạo ra các “lỗ trống”, hình thành cấu trúc fractal đặc trưng.

```
function generateTriangle(depth) {
  const vertices = [], colors = [];
  const height = 1.6 * Math.sqrt(3) / 2;
  const v1 = [-0.8, -height / 3], v2 = [0.8, -height / 3], v3 = [0, 2 * height / 3];
  const midpoint = (a, b) => [(a[0] + b[0]) / 2, (a[1] + b[1]) / 2];

  function subdivide(p1, p2, p3, d) {
    if (d === 0) {
      vertices.push(p1[0], p1[1], p2[0], p2[1], p3[0], p3[1]);
      const color = rainbowColor((p1[0] + p2[0] + p3[0]) / 3, (p1[1] + p2[1] + p3[1]) / 3);
      colors.push(...color, ...color, ...color);
      return;
    }
    const m12 = midpoint(p1, p2), m23 = midpoint(p2, p3), m31 = midpoint(p3, p1);
    subdivide(p1, m12, m31, d - 1);
    subdivide(m12, p2, m23, d - 1);
    subdivide(m31, m23, p3, d - 1);
  }

  subdivide(v1, v2, v3, depth);
  return { vertices, colors, triangleCount: Math.pow(3, depth), vertexCount: vertices.length / 2 };
}
```

2.3.2. Thảm Sierpinski (Sierpinski Carpet):

2.3.2.1. Định nghĩa và phương pháp xây dựng:

Thảm Sierpinski là phiên bản hai chiều mở rộng của ý tưởng fractal trên hình vuông:

- 1. **Bước 0:** Bắt đầu với một hình vuông.
- 2. **Bước 1:** Chia hình vuông thành lưới 3×3 gồm 9 ô bằng nhau, sau đó loại bỏ ô ở chính giữa.
- 3. **Bước n:** Áp dụng lại quy trình trên cho từng ô vuông còn lại.

2.3.2.2. Số lượng hình vuông và số đỉnh:

Số lượng hình vuông tăng theo cấp số nhân với cơ số 8:

Bước (n):	Số hình vuông:	Số đỉnh (WebGL):
0	1	6
1	8	48
2	64	384
n	8^n	6×8^n

Lưu ý: Mỗi hình vuông được biểu diễn bằng 2 tam giác trong WebGL, tương ứng với 6 đỉnh.

2.3.2.3. Thuật toán đệ quy:

Tại mỗi bước, ô vuông trung tâm bị loại bỏ. Quá trình đệ quy tiếp tục được áp dụng cho 8 ô vuông còn lại, tạo nên cấu trúc fractal dạng lưới đặc trưng.

```
function generateCarpet(depth) {
  const vertices = [], colors = [];
  function subdivide(x, y, s, d) {
    if (d === 0) {
      vertices.push(x, y, x + s, y, x, y + s, x + s, y, x + s, y + s, x, y + s);
      const color = rainbowColor(x + s / 2, y + s / 2);
      for (let i = 0; i < 6; i++) colors.push(...color);
      return;
    }
    const ns = s / 3;
    for (let i = 0; i < 3; i++) for (let j = 0; j < 3; j++) if (!(i === 1 && j === 1))
      subdivide(x + i * ns, y + j * ns, ns, d - 1);
  }
  subdivide(-0.85, -0.85, 1.7, depth);
  return { vertices, colors, squareCount: Math.pow(8, depth), vertexCount: vertices.length / 2 };
}
```

2.3.3. Nhận xét và đánh giá:

* Điểm giống nhau:

- Được xây dựng bằng thuật toán đệ quy.
- Diện tích tiến dần về 0 khi số bước tăng & có chiều fractal nằm giữa 1 và 2.

* Điểm khác nhau:

Tiêu chí:	Tam giác Sierpinski:	Thảm Sierpinski:
Hình nền:	Tam giác đều	Hình vuông.
Cách chia:	4 phần (loại bỏ 1)	Lưới 3×3 (loại bỏ 1)
Hệ số tăng:	3	8
Chiều fractal:	≈ 1.585	≈ 1.893
Độ phức tạp:	$O(3^n)$	$O(8^n)$

2.4. Tìm hiểu và trình bày tập Mandelbrot và Julia Set:

2.4.1. Cơ sở toán học và Hệ động lực phức:

Cả tập Mandelbrot và tập Julia đều được sinh ra từ một hàm số phức lặp lại (iterative function) có dạng: $f_c(z) = z^2 + c$

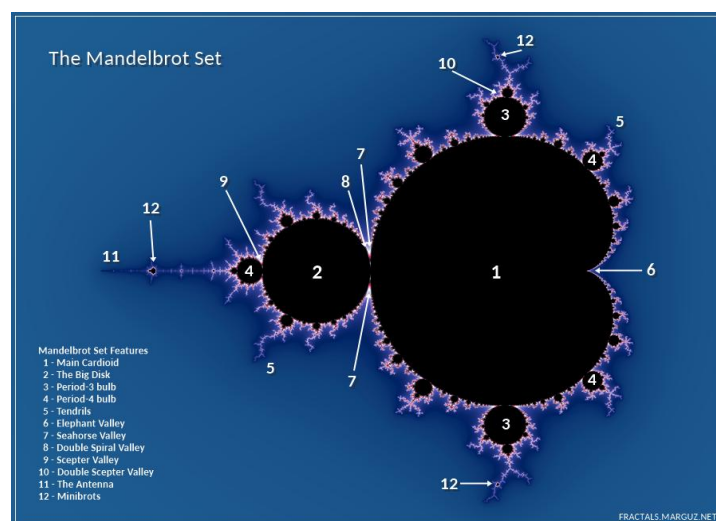
Trong đó z và c là các số phức ($z = x + iy$). Trong đồ họa máy tính, mỗi điểm trên màn hình (x, y) sẽ được ánh xạ thành một điểm trên mặt phẳng phức.

- **Quá trình lặp:** Từ một giá trị khởi đầu z_0 , ta tính $z_1 = f(z_0), z_2 = f(z_1), \dots, z_n = f(z_{n-1})$.
- **Sự hội tụ và phân kỳ:** Nếu quỹ đạo của z tiến ra vô cực, điểm đó nằm ngoài tập hợp. Nếu quỹ đạo của z bị giới hạn trong một khoảng nhất định (thường là bán kính $R = 2$), điểm đó thuộc về tập hợp.

2.4.2. Tập hợp Mandelbrot:

Định nghĩa: Tập Mandelbrot là tập hợp các giá trị c trong mặt phẳng phức sao cho dãy số lặp bắt đầu từ $z_0 = 0$ không tiến ra vô cùng.

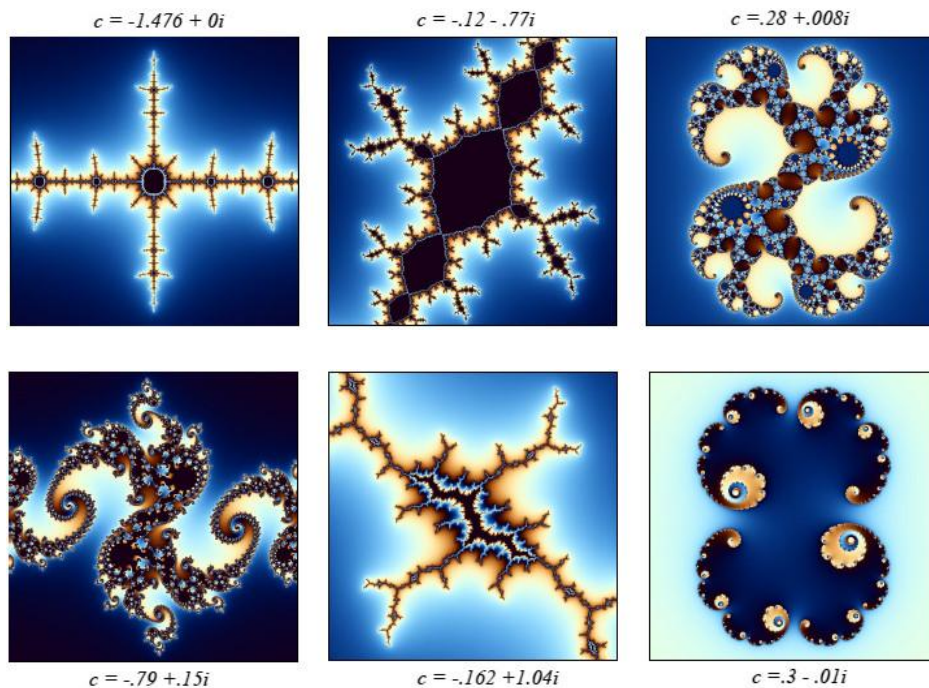
- **Đặc điểm:** Đây là một cấu trúc Fractal cực kỳ phức tạp. Càng phóng to vào biên của tập Mandelbrot, ta càng thấy các cấu trúc tự đồng dạng (self-similar) vô tận.
- **Ý nghĩa:** Nó đóng vai trò là "bản đồ chỉ số" cho các tập Julia.



2.4.3. Tập hợp Julia:

Định nghĩa: Với mỗi giá trị c cố định, tập Julia là tập hợp các điểm khởi đầu z_0 sao cho dãy số lặp không tiến ra vô cùng.

- **Sự đa dạng:** Với mỗi số phức c khác nhau, ta có một tập Julia hoàn toàn khác nhau.
- **Tập Julia đầy đủ (Filled Julia Set):** Là phần diện tích bên trong biên của tập Julia. Trong WebGL, chúng ta thường vẽ biên này bằng cách tô màu dựa trên tốc độ phân kỳ của các điểm.



2.4.4. Mối quan hệ giữa Tập hợp Mandelbrot và Tập hợp Julia:

Mặc dù có hình thái khác nhau, tập Mandelbrot và tập Julia thực chất là hai cách nhìn khác nhau về cùng một hệ động lực phức được xác định bởi hàm số $f_c(z) = z^2 + c$. Mối quan hệ này có thể được hiểu qua các khía cạnh sau:

a. Mặt phẳng Tham số & Mặt phẳng Động:

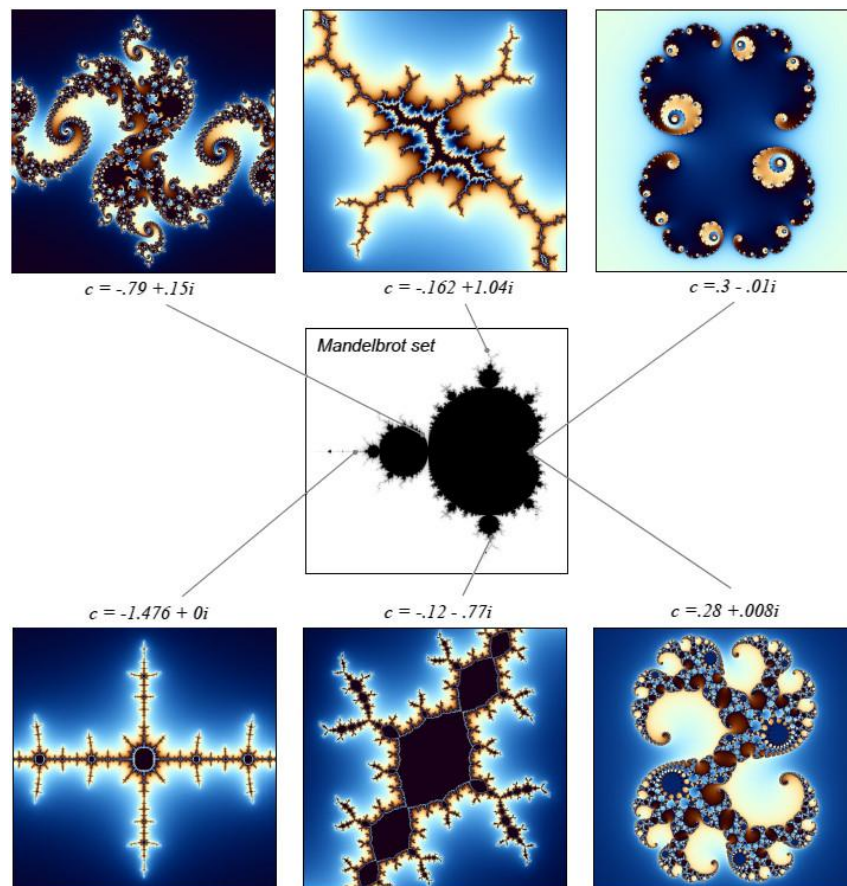
- Tập Mandelbrot (Mặt phẳng Tham số): Đóng vai trò là một "bản đồ chỉ số" (index map). Mỗi điểm c trên mặt phẳng này đại diện cho một tập Julia duy nhất.

- Tập Julia (Mặt phẳng Động): Là "không gian trạng thái" của một giá trị c cụ thể. Với mỗi điểm c được chọn từ tập Mandelbrot, chúng ta sẽ có một hình ảnh tập Julia tương ứng trên mặt phẳng phức.

b. Định lý về tính liên thông (Connectivity Theorem):

Một trong các mối quan hệ quan trọng được chứng minh bởi Gaston Julia và Pierre Fatou là:

- Nếu giá trị c nằm bên trong hoặc trên biên của tập Mandelbrot ($c \in M$), thì tập Julia tương ứng là một tập hợp liên thông (không bị đứt gãy).
- Nếu giá trị c nằm ngoài tập Mandelbrot ($c \notin M$), thì tập Julia tương ứng sẽ bị phân rã thành vô số các điểm rời rạc, tạo thành một cấu trúc gọi là "bụi Cantor" (Cantor dust).



c. Phân tích sự biến đổi hình thái (Dựa trên Hình 3):

- **Vùng trung tâm (Main Cardioids):** Khi c nằm sâu trong các thùy chính của tập Mandelbrot (ví dụ: $c = -0.12 - 0.77i$), tập Julia có xu hướng là các khối đặc, tròn trịa và liên kết chặt chẽ.

- Vùng biên và các nhánh cây (Boundary & Filaments):** Khi di chuyển c ra sát biên phức tạp của tập Mandelbrot, tập Julia bắt đầu xuất hiện các cấu trúc phân mảnh, gai góc và mang tính tự đồng dạng cực cao (ví dụ: $c = -0.79 + 0.15i$ - cấu trúc xoắn ốc).
- Vùng đuôi (The Needle):** Các giá trị c nằm trên trục thực âm (ví dụ: $c = -1.476$) tạo ra các tập Julia có dạng hình học đối xứng và trải dài theo trục.

Kết luận: Tập Mandelbrot chứa đựng tất cả các hình thái có thể có của tập Julia. Việc nghiên cứu tập Mandelbrot cho phép các nhà toán học và lập trình viên dự đoán trước được cấu trúc của một tập Julia bất kỳ mà không cần phải dựng toàn bộ hình ảnh của nó.

*** Bảng tổng hợp các đặc điểm phân biệt giữa tập hợp Mandelbrot và tập hợp Julia:**

Đặc điểm so sánh:	Tập hợp Mandelbrot (M-set):	Tập hợp Julia (J-set):
Công thức truy hồi:	$z_{n+1} = z_n^2 + c$	$z_{n+1} = z_n^2 + c$
Vai trò của c:	Biến số chính: Thay đổi theo tọa độ mỗi pixel trên mặt phẳng phức.	Tham số cố định: Được chọn trước và giữ nguyên cho toàn bộ hình.
Giá trị khởi đầu z_0 :	Luôn cố định tại gốc tọa độ: $z_0 = 0 + 0i.$	Biến số: Là tọa độ phức tương ứng của từng pixel trên màn hình.
Không gian biểu diễn:	Mặt phẳng tham số (Parameter Plane): Mỗi điểm đại diện cho một hệ động lực.	Mặt phẳng động (Dynamical Plane): Biểu diễn hành vi của các điểm dưới một hệ động lực cố định.
Tính duy nhất:	Chỉ có một tập Mandelbrot duy nhất trong toán học.	Có vô số tập Julia khác nhau tùy thuộc vào giá trị c được chọn.
Tính tự đồng dạng:	Quasi-self-similarity: Các cấu trúc lặp lại nhưng có biến đổi, không giống hệt nhau hoàn toàn.	Strict self-similarity: Có tính tự đồng dạng rất cao, các chi tiết lặp lại chính xác ở mọi cấp độ.
Tính liên thông (Topology):	Luôn luôn là một tập hợp liên thông (một khối duy nhất)	Có thể liên thông (nếu $c \in M$) hoặc là các điểm rời rạc (nếu $c \notin M$).

Hình dáng đặc trưng:	Hình tim (Cardioid) chủ đạo với các thùy tròn xung quanh (Bulbs).	Đa dạng: Hình xoắn ốc, hình hạt bụi, hình "con rồng",...
Mục đích sử dụng:	Dùng làm bản đồ tra cứu để tìm các vùng có tập Julia đẹp.	Dùng để tạo ra các tác phẩm nghệ thuật Fractal có cấu trúc đối xứng.

2.4.5. Thuật toán Escape Time (Thời gian thoát):

Để hiển thị các tập hợp này trên màn hình máy tính bằng WebGL, thuật toán Escape Time được sử dụng:

- Với mỗi Pixel (px, py), chuyển đổi nó sang tọa độ phức (x, y).
- Thực hiện vòng lặp tính $z = z^2 + c$ tối đa N lần (ví dụ N = 100 hoặc 1000).
- Nếu tại bước lặp thứ i, độ lớn $|z| = \sqrt{x^2 + y^2} > 2$, ta dừng lại và dùng giá trị i để quyết định màu sắc (điểm thoát nhanh có màu khác điểm thoát chậm).
- Nếu sau N lần lặp mà $|z|$ vẫn ≤ 2 , ta tô màu đen (điểm thuộc tập hợp).

2.4.6. Các kỹ thuật tối ưu hóa và nâng cao chất lượng hiển thị:

a. Khử răng cưa (Anti-aliasing):

Trong WebGL, thay vì chỉ lấy 1 mẫu tại tâm pixel, ta có thể lấy mẫu tại 4 hoặc 9 điểm xung quanh (Super-sampling) rồi lấy trung bình cộng để hình ảnh Fractal mịn màng hơn ở các vùng biên phức tạp.

b. Tô màu mượt mà (Smooth Coloring / Renormalization):

Thay vì tô màu theo số nguyên i (gây hiện tượng phân tầng màu - banding), ta sử dụng công thức nội suy dựa trên logarit để có dải màu chuyển tiếp mượt mà:

$$v = i + 1 - \log_2(\log_2(|z|))$$

Giá trị v này sẽ cho ra màu sắc liên tục, tạo hiệu ứng thị giác chuyên nghiệp.

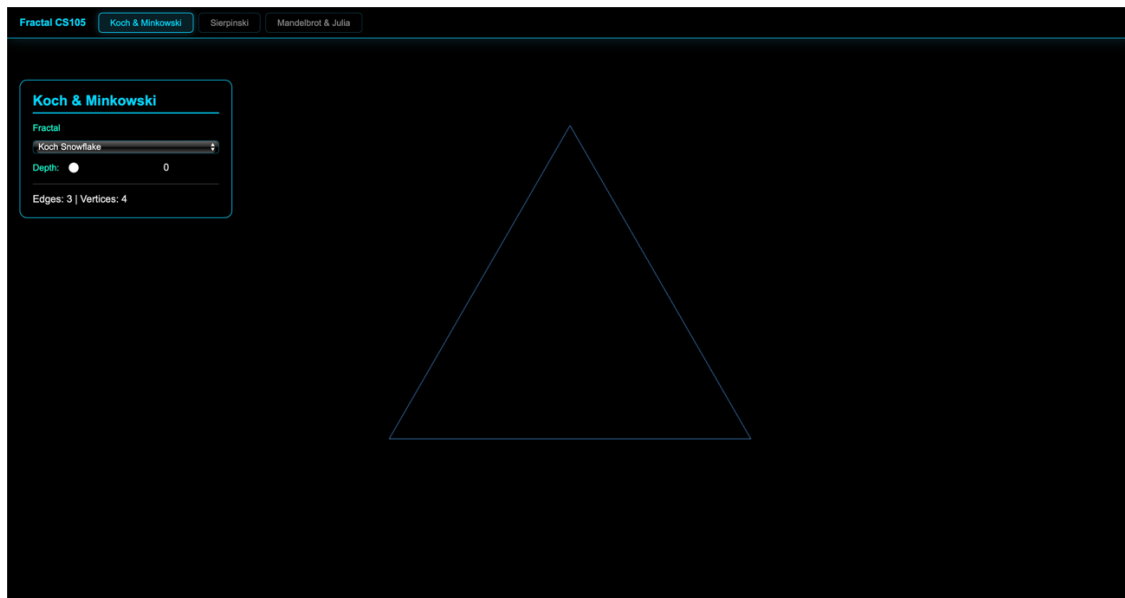
3. Demo:

Để minh họa trực quan cho các khái niệm lý thuyết, nhóm đã xây dựng một ứng dụng web tích hợp các mô phỏng Fractal sử dụng công nghệ WebGL. Ứng dụng cho phép tương tác thời gian thực, giúp quan sát rõ nét tính tự đồng dạng và quy trình hình thành của các cấu trúc Fractal thông qua việc tính toán song song trên GPU.

3.1. Giao diện tổng quan:

Giao diện ứng dụng được thiết kế theo phong cách hiện đại với bảng điều khiển nằm ở phía bên trái màn hình.

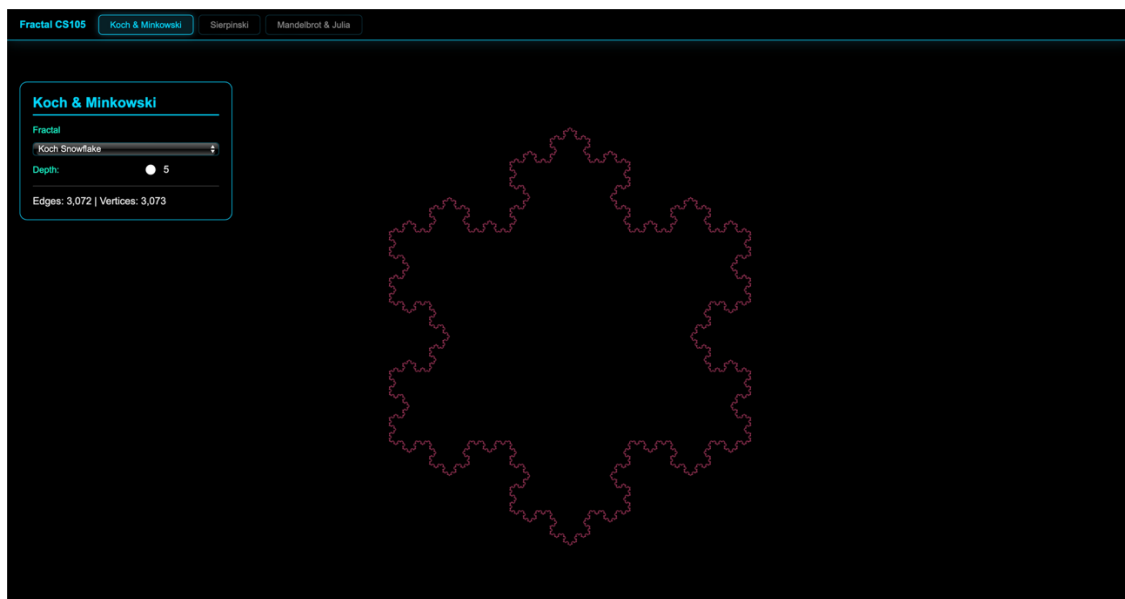
- **Công nghệ lõi:** Sử dụng WebGL để giao tiếp trực tiếp với nhân đồ họa (GPU).
- **Xử lý thuật toán:** Các thuật toán Fractal được viết bằng ngôn ngữ GLSL (Fragment Shader) để đảm bảo tốc độ tính toán hàng triệu pixel trong thời gian thực.
- **Quản lý giao diện:** Sử dụng HTML5/CSS3 và JavaScript để quản lý việc chuyển đổi giữa các loại Fractal khác nhau.



3.2. Chi tiết các phân hệ Fractal và Hướng dẫn tương tác:

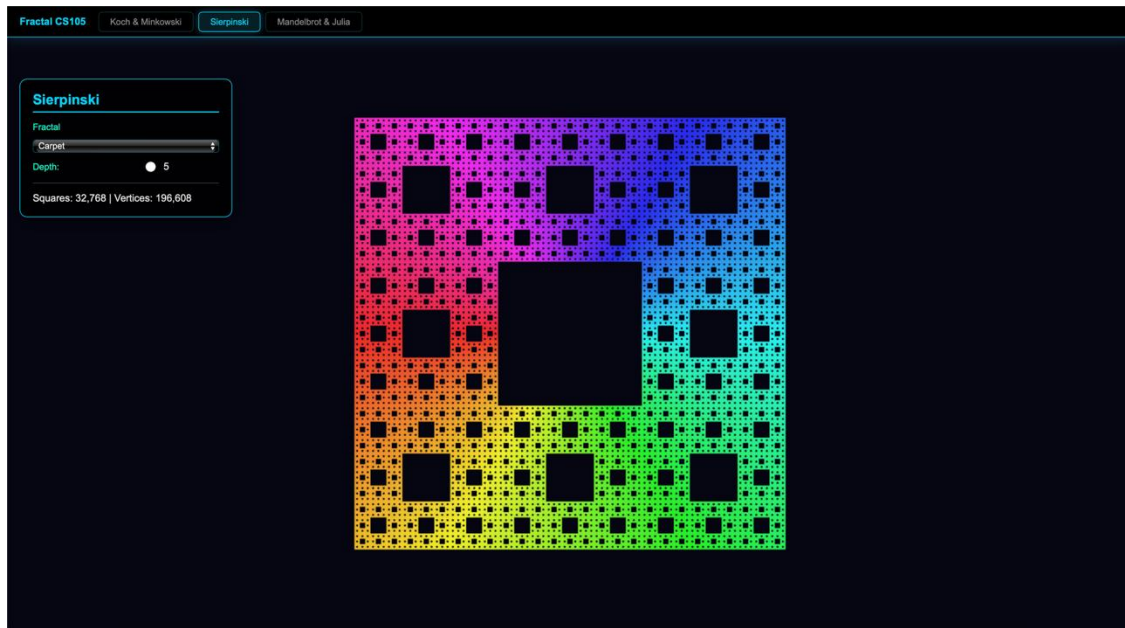
3.2.1. Phân hệ Koch & Minkowski:

- **Công dụng:** Trực quan hóa quy trình đệ quy tạo nên đường cong Koch và đảo Minkowski.
- **Tính năng:** * Chọn loại Fractal (Koch Snowflake hoặc Minkowski Island) từ danh sách thả xuống.
 - Thay đổi cấp độ đệ quy (Depth) từ 0 đến 5 bằng thanh trượt.
- **Thông tin hiển thị:** Số lượng cạnh (Edges) và số đỉnh (Vertices) thực tế được vẽ trên màn hình.



3.2.2. Phân hệ Sierpinski (Triangle & Carpet):

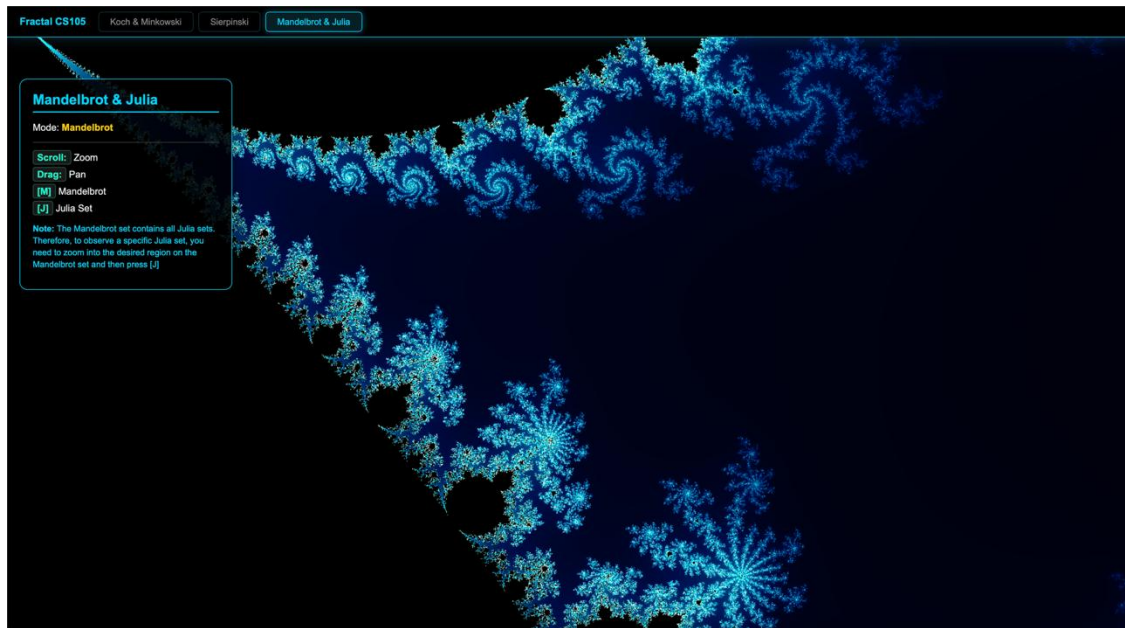
- **Công dụng:** Minh họa tính chất tự đồng dạng và sự suy giảm diện tích của Fractal dạng diện tích.
- **Đặc điểm hình thái:** Sử dụng thuật toán tô màu cầu vồng (Rainbow Color) dựa trên tọa độ trung điểm để tạo hiệu ứng thị giác rực rỡ.
- **Thông tin hiển thị:** Thống kê số lượng tam giác con hoặc hình vuông con được sinh ra sau mỗi bước lặp.



3.2.3. Phân hệ Mandelbrot & Julia Set:

Đây là phần có tính tương tác cao nhất, thể hiện mối quan hệ khăng khít giữa mặt phẳng tham số và mặt phẳng động .

- **Tương tác chuột: * Cuộn chuột (Scroll):** Phóng to/Thu nhỏ vào các vùng biên phức tạp (Filaments).
 - **Kéo chuột (Drag):** Di chuyển trên mặt phẳng phức để tìm kiếm các cấu trúc hình học đẹp.
- **Phím tắt điều khiển:**
 - **Phím [M]:** Xem tập Mandelbrot – đóng vai trò là "bản đồ chỉ số".
 - **Phím [J]:** Xem tập Julia tương ứng với vị trí con trỏ chuột hiện tại trên tập Mandelbrot.
 - **Phím [R]:** Đặt lại góc nhìn mặc định cho chế độ hiện tại.
- **Kỹ thuật xử lý:** Áp dụng thuật toán **Smooth Coloring** (nội suy logarit) để loại bỏ hiện tượng phân tầng màu, tạo ra dải màu chuyển tiếp mượt mà rực rỡ (Neon Blue).



3.3. Hiệu quả thực thi:

Nhờ việc tận dụng Fragment Shader, ứng dụng có thể duy trì tốc độ khung hình ổn định (60 FPS) ngay cả khi người dùng thực hiện các thao tác phóng đại sâu vào các vùng có cấu trúc phức tạp. Hệ thống lưu giữ trạng thái (State Persistence) giúp người dùng không bị mất dấu vị trí khi chuyển đổi qua lại giữa tập Mandelbrot và Julia, hỗ trợ đắc lực cho việc nghiên cứu sự tương quan hình thái giữa hai tập hợp này.

3.4. Quy trình Đóng gói Ứng dụng (.exe) với Node.js

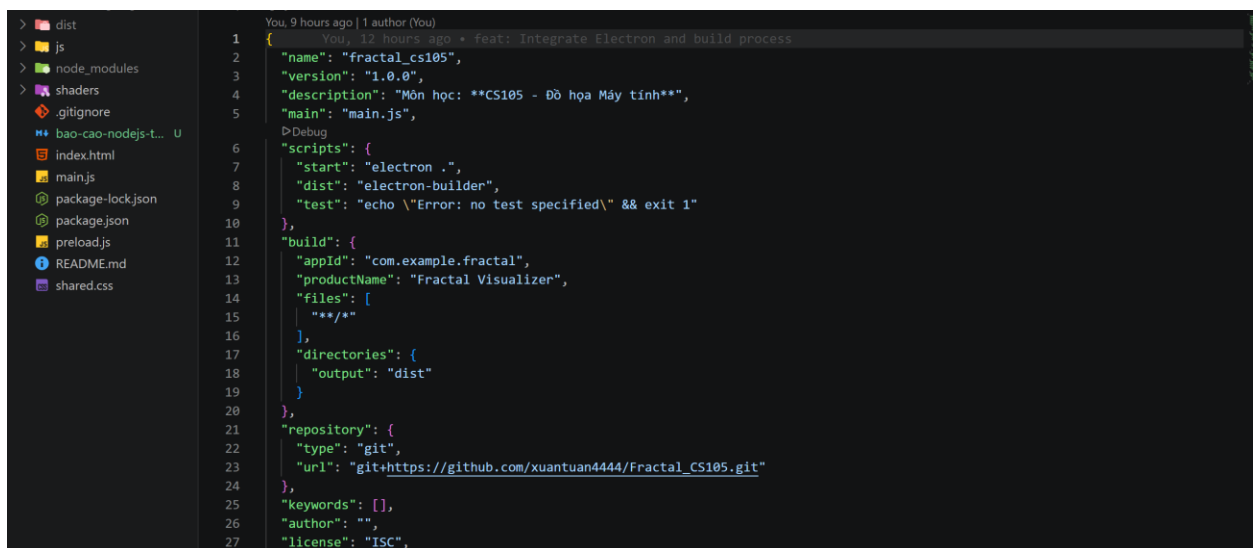
3.4.1. Kiến trúc Công cụ

Dự án ứng dụng Node.js để chuyển đổi mã nguồn web thành ứng dụng Windows Desktop độc lập, thông qua hai package lõi:

- **Electron (-v 41.2.0):** Cung cấp môi trường runtime (kết hợp Chromium và Node.js) để render giao diện.
- **electron-builder (-v 26.8.1):** Trình biên dịch và đóng gói tự động xuất bản file cài đặt.

3.4.2. Cấu hình Build (package.json)

Các tham số cấu hình được tích hợp trực tiếp, định nghĩa siêu dữ liệu ứng dụng và kịch bản thực thi:



```
1 {
2   "name": "fractal_cs105",
3   "version": "1.0.0",
4   "description": "Môn học: **CS105 - Đồ họa Máy tính**",
5   "main": "main.js",
6   "scripts": {
7     "start": "electron .",
8     "dist": "electron-builder",
9     "test": "echo \\\"Error: no test specified\\\" && exit 1"
10  },
11  "build": {
12    "appId": "com.example.fractal",
13    "productName": "Fractal Visualizer",
14    "files": [
15      "**/*"
16    ],
17    "directories": {
18      "output": "dist"
19    }
20  },
21  "repository": {
22    "type": "git",
23    "url": "git+https://github.com/xuantuan4444/Fractal_CS105.git"
24  },
25  "keywords": [],
26  "author": "",
27  "license": "ISC",
```

3.4.3. Quy trình Thực thi

Việc triển khai diễn ra thông qua giao diện dòng lệnh (CLI) với 3 bước tiêu chuẩn:

1. **Khởi tạo môi trường:** npm install (Tải các dependencies cần thiết).
2. **Kiểm thử cục bộ (Local Testing):** npm start (Đảm bảo ứng dụng chạy ổn định trước khi đóng gói).
3. **Build sản phẩm:** npm run dist (Tiến hành biên dịch và đóng gói).

3.4.4. Kết quả Đầu ra (Thư mục dist/)

Quá trình build hoàn tất sẽ tự động tạo thư mục dist/ chứa các tệp tin phát hành:

- **Bộ cài đặt chính thức (Installer):** Fractal Visualizer Setup 1.0.0.exe (Dùng để phân phối cho người dùng cuối).
- **Bản thực thi trực tiếp (Portable):** Thư mục win-unpacked/ (Có thể chạy ứng dụng ngay không cần cài đặt).
- **Dữ liệu phụ trợ:** File .blockmap và các file cấu hình .yaml (Phục vụ cho tính năng Auto-update và debug).